13.3 会话生命周期

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~250,000 tokens，输出 ~18,000 tokens（本章合计）

会话不是静态的数据容器——它有创建、活跃、过期和重置的完整生命周期。OpenClaw 通过灵活的重置策略控制会话的新旧交替，并提供运行时修补（Patch）机制让用户和 Agent 可以动态调整会话属性。本节分析会话生命周期管理的核心源码。

13.3.1 创建与初始化：首条消息触发会话创建

OpenClaw 的会话采用**懒创建**（Lazy Creation）策略：没有预先创建会话这一步，第一条消息到达时自动创建。 `src/auto-reply/reply/session.ts` 中的 `initSessionState()` 函数是会话初始化的核心入口。

```ts
// src/auto-reply/reply/session.ts
export async function initSessionState(params: {
  ctx: MsgContext;
  cfg: OpenClawConfig;
  commandAuthorized: boolean;
}): Promise<SessionInitResult> {
  const { ctx, cfg } = params;
  const sessionCfg = cfg.session;

  // 1. 加载会话存储
  const storePath = resolveStorePath(sessionCfg?.store, { agentId });
  const sessionStore: Record<string, SessionEntry> = loadSessionStore(storePath);

  // 2. 解析会话键
  sessionKey = resolveSessionKey(sessionScope, sessionCtxForState, mainKey);
  const entry = sessionStore[sessionKey];

  // 3. 判断是否为新会话
  const freshEntry = entry
    ? evaluateSessionFreshness({ updatedAt: entry.updatedAt, now, policy: resetPolicy }
    : false;

  if (!isNewSession && freshEntry) {
    // 复用已有会话
    sessionId = entry.sessionId;
    systemSent = entry.systemSent ?? false;
  } else {
    // 创建新会话
    sessionId = crypto.randomUUID();
    isNewSession = true;
    systemSent = false;
  }
  // ...
}
```

这个流程的关键在于第 3 步的"新鲜度"判断：如果会话键在存储中不存在（`entry` 为 `undefined`），`freshEntry` 为 `false`，会话自动创建；如果存在但已过期（不再"新鲜"），同样创建新会话。

会话被标记为新会话时，系统会重置一系列状态：

```
// src/auto-reply/reply/session.ts（新会话初始化）
if (isNewSession) {
  sessionEntry.compactionCount = 0;          // 重置压缩计数
  sessionEntry.memoryFlushCompactionCount = undefined;
  sessionEntry.memoryFlushAt = undefined;
  sessionEntry.totalTokens = undefined;       // 清除 Token 用量
  sessionEntry.inputTokens = undefined;
  sessionEntry.outputTokens = undefined;
  sessionEntry.contextTokens = undefined;
}
```

这确保新会话从干净的状态开始，不会继承旧会话的 Token 统计和压缩状态。

重置触发器（Reset Triggers）

用户可以通过发送特定命令（如 `/new` 或 `/reset`）手动触发会话重置，这些命令称为**重置触发器**：

```
// src/auto-reply/reply/session.ts
const resetTriggers = sessionCfg?.resetTriggers?.length
  ? sessionCfg.resetTriggers
  : DEFAULT_RESET_TRIGGERS;  // 默认 ["/new", "/reset"]

for (const trigger of resetTriggers) {
  if (!resetAuthorized) break;   // 未授权用户无法重置
  const triggerLower = trigger.toLowerCase();
  if (trimmedBodyLower === triggerLower || strippedForResetLower === triggerLower) {
    isNewSession = true;
    bodyStripped = "";
    resetTriggered = true;
    break;
  }
  // 也支持 "/new 你好" 这样的带消息重置
  const triggerPrefixLower = `${triggerLower} `;
  if (trimmedBodyLower.startsWith(triggerPrefixLower)) {
    isNewSession = true;
    bodyStripped = strippedForReset.slice(trigger.length).trimStart();
    resetTriggered = true;
    break;
  }
}
```

触发器匹配是**大小写不敏感**的（用户输入 `/NEW` 也能匹配），并且支持在重置命令后附加消息（如 `/new 帮我写个邮件` ），此时会话重置后这条附加消息会作为新会话的第一条消息。

13.3.2 重置策略：每日重置（Daily Reset）与空闲重置（Idle Reset）

除了手动重置，OpenClaw 还支持两种自动重置策略。 `src/config/sessions/reset.ts` 定义了相关类型和逻辑。

```
// src/config/sessions/reset.ts
export type SessionResetMode = "daily" | "idle";

export type SessionResetPolicy = {
  mode: SessionResetMode;
  atHour: number;          // 每日重置的小时（0-23）
  idleMinutes?: number;    // 空闲超时分钟数
};

export const DEFAULT_RESET_MODE: SessionResetMode = "daily";
export const DEFAULT_RESET_AT_HOUR = 4;  // 默认凌晨 4 点
```

每日重置（Daily Reset）

每日重置是**默认策略**。语义是：如果会话的最后更新时间早于今天的重置时间点，则会话被视为过期。

```
// src/config/sessions/reset.ts
export function resolveDailyResetAtMs(now: number, atHour: number): number {
  const resetAt = new Date(now);
  resetAt.setHours(normalizeResetAtHour(atHour), 0, 0, 0);
  // 如果当前时间还没到今天的重置时间，则使用昨天的重置时间
  if (now < resetAt.getTime()) {
    resetAt.setDate(resetAt.getDate() - 1);
  }
  return resetAt.getTime();
}
```

举例：假设重置时间为凌晨 4 点：

- 现在是 2 月 18 日上午 10 点 → 重置时间线是 2 月 18 日 04:00
- 现在是 2 月 18 日凌晨 2 点 → 重置时间线是 2 月 17 日 04:00（因为还没过今天的 4 点）

任何在重置时间线**之前**最后更新的会话都被视为过期。

空闲重置（Idle Reset）

空闲重置基于会话的最后活跃时间。如果会话在指定分钟数内没有新消息，则视为过期。

新鲜度评估

`evaluateSessionFreshness()` 是判断会话是否过期的核心函数：

```
// src/config/sessions/reset.ts
export function evaluateSessionFreshness(params: {
  updatedAt: number;
  now: number;
  policy: SessionResetPolicy;
}): SessionFreshness {
  // 计算每日重置时间线（仅在 daily 模式下）
  const dailyResetAt = params.policy.mode === "daily"
    ? resolveDailyResetAtMs(params.now, params.policy.atHour)
    : undefined;
  // 计算空闲过期时间（如果配置了 idleMinutes）
  const idleExpiresAt = params.policy.idleMinutes != null
    ? params.updatedAt + params.policy.idleMinutes * 60_000
    : undefined;
  // 两种过期条件取 OR：任一满足即过期
  const staleDaily = dailyResetAt != null && params.updatedAt < dailyResetAt;
  const staleIdle = idleExpiresAt != null && params.now > idleExpiresAt;
  return {
    fresh: !(staleDaily || staleIdle),
    dailyResetAt,
    idleExpiresAt,
  };
}
```

一个关键设计：**两种过期条件可以同时生效**。即使处于 `daily` 模式，如果同时配置了 `idleMinutes`，空闲超时也会触发过期。这意味着可以设置"每天凌晨 4 点重置，但如果空闲超过 2 小时也重置"这样的组合策略。

13.3.3 按类型/通道覆盖重置策略

不同类型的会话可能需要不同的重置策略。DM 对话可能适合每日重置，而群组中的线程会话可能更适合短时间空闲后重置。OpenClaw 通过 `resetByType` 和 `resetByChannel` 提供了细粒度的覆盖机制。

会话类型分类

`resolveSessionResetType()` 根据会话键和消息属性将会话分为三类：

```
// src/config/sessions/reset.ts
export type SessionResetType = "dm" | "group" | "thread";

export function resolveSessionResetType(params: {
  sessionKey?: string | null;
  isGroup?: boolean;
  isThread?: boolean;
}): SessionResetType {
  // 优先级：thread > group > dm
  if (params.isThread || isThreadSessionKey(params.sessionKey)) return "thread";
  if (params.isGroup) return "group";
  const normalized = (params.sessionKey ?? "").toLowerCase();
  if (GROUP_SESSION_MARKERS.some((marker) => normalized.includes(marker))) return "grou
  return "dm";
}
```

策略解析优先级

`resolveSessionResetPolicy()` 按以下优先级解析重置策略：

```ts
// src/config/sessions/reset.ts
export function resolveSessionResetPolicy(params: {
  sessionCfg?: SessionConfig;
  resetType: SessionResetType;
  resetOverride?: SessionResetConfig;  // 来自 resetByChannel 的覆盖
}): SessionResetPolicy {
  const baseReset = params.resetOverride ?? sessionCfg?.reset;
  const typeReset = params.resetOverride
    ? undefined
    : sessionCfg?.resetByType?.[params.resetType];
  // 优先级：resetByChannel > resetByType > reset（基础配置）
  const mode = typeReset?.mode ?? baseReset?.mode ?? DEFAULT_RESET_MODE;
  const atHour = typeReset?.atHour ?? baseReset?.atHour ?? DEFAULT_RESET_AT_HOUR;
  // ...
}
```

以下配置示例展示了这种层次化的覆盖机制：

```json
// openclaw.json5
{
  "session": {
    "reset": { "mode": "daily", "atHour": 4 },
    "resetByType": {
      "thread": { "mode": "idle", "idleMinutes": 180 }   // 线程：3 小时空闲重置
    },
    "resetByChannel": {
      "discord": { "mode": "idle", "idleMinutes": 10080 } // Discord：7 天空闲重置
    }
  }
}
```

在这个配置下：

- **DM 会话**：使用默认策略，每天凌晨 4 点重置
- **线程会话**：3 小时无活动后重置（由 `resetByType.thread` 覆盖）
- **Discord 上的所有会话**：7 天无活动后重置（由 `resetByChannel.discord` 覆盖，优先级高于 `resetByType`）

`resolveChannelResetConfig()` 负责查找通道特定的覆盖配置：

```
// src/config/sessions/reset.ts
export function resolveChannelResetConfig(params: {
  sessionCfg?: SessionConfig;
  channel?: string | null;
}): SessionResetConfig | undefined {
  const resetByChannel = params.sessionCfg?.resetByChannel;
  if (!resetByChannel) return undefined;
  const key = normalizeMessageChannel(params.channel) ?? params.channel?.trim().toLower
  if (!key) return undefined;
  return resetByChannel[key] ?? resetByChannel[key.toLowerCase()];
}
```

13.3.4 会话修补（Session Patch）：运行时修改会话属性

会话一旦创建，属性并非一成不变。用户可以通过 `sessions.patch` 协议方法在运行时修改会话的模型、思考级别、发送策略等属性，而无需重置整个会话。

`src/gateway/sessions-patch.ts` 中的 `applySessionsPatchToStore()` 函数实现了这一机制：

```
// src/gateway/sessions-patch.ts
export async function applySessionsPatchToStore(params: {
  cfg: OpenClawConfig;
  store: Record<string, SessionEntry>;
  storeKey: string;
  patch: SessionsPatchParams;
  loadGatewayModelCatalog?: () => Promise<ModelCatalogEntry[]>;
}): Promise<{ ok: true; entry: SessionEntry } | { ok: false; error: ErrorShape }> {
  const existing = store[storeKey];
  // 存在则更新，不存在则创建
  const next: SessionEntry = existing
    ? { ...existing, updatedAt: Math.max(existing.updatedAt ?? 0, now) }
    : { sessionId: randomUUID(), updatedAt: now };
  // ...逐字段应用 patch...
  store[storeKey] = next;
  return { ok: true, entry: next };
}
```

可修补的属性

会话修补支持以下属性：

属性	类型	说明
`label`	`string | null`	会话标签（必须唯一）
`thinkingLevel`	`string | null`	思考级别（low/medium/high/xhigh）
`verboseLevel`	`string | null`	详细输出级别
`reasoningLevel`	`string | null`	推理级别（on/off/stream）
`responseUsage`	`string | null`	用量显示（off/tokens/full）
`model`	`string | null`	模型覆盖（null 恢复默认）
`sendPolicy`	`string | null`	发送策略（allow/deny）
`execHost`	`string | null`	执行宿主（sandbox/gateway/node）
`execSecurity`	`string | null`	执行安全级别（deny/allowlist/full）
`groupActivation`	`string | null`	群组激活方式（mention/always）

每个字段都有严格的校验逻辑。以模型覆盖为例：

```
// src/gateway/sessions-patch.ts（模型修补片段）
if ("model" in patch) {
  const raw = patch.model;
  if (raw === null) {
    // null 表示恢复默认模型
    applyModelOverrideToSessionEntry({
      entry: next,
      selection: { provider: resolvedDefault.provider, model: resolvedDefault.model, is
    });
  } else if (raw !== undefined) {
    const trimmed = String(raw).trim();
    if (!trimmed) return invalid("invalid model: empty");
    // 需要从模型目录中验证模型是否合法
    const catalog = await params.loadGatewayModelCatalog();
    const resolved = resolveAllowedModelRef({ cfg, catalog, raw: trimmed, ... });
    if ("error" in resolved) return invalid(resolved.error);
    applyModelOverrideToSessionEntry({ entry: next, selection: { ... } });
  }
}
```

修补机制的一个重要特性是**设置为 `null` 表示清除覆盖**（恢复默认值），而 `undefined` （即 patch 中不包含该字段）表示不修改。这遵循了 JSON Patch 中常见的语义约定。

服务端会话重置

Gateway 的 `sessions.reset` 方法提供了服务端的完整重置能力：

```
// src/gateway/server-methods/sessions.ts
"sessions.reset": async ({ params, respond }) => {
  const next = await updateSessionStore(storePath, (store) => {
    const entry = store[primaryKey];
    const nextEntry: SessionEntry = {
      sessionId: randomUUID(),       // 生成新的会话 ID
      updatedAt: now,
      systemSent: false,
      abortedLastRun: false,
      // 保留用户的个性化设置
      thinkingLevel: entry?.thinkingLevel,
      verboseLevel: entry?.verboseLevel,
      model: entry?.model,
      sendPolicy: entry?.sendPolicy,
      label: entry?.label,
      // 重置 Token 计数
      inputTokens: 0, outputTokens: 0, totalTokens: 0,
    };
    store[primaryKey] = nextEntry;
    return nextEntry;
  });
};
```

重置时会**保留个性化覆盖**（如模型选择、思考级别），但**清除对话状态**（生成新的 session ID、重置 Token 计数）。这样用户在 `/reset` 后不需要重新配置偏好设置。

> **v2026.3.9 更新：会话重置服务拆分**
>
> v2026.3.9 引入了 `session-reset-service.ts` （ `src/gateway/session-reset-service.ts` ），将用户发起的 `/new` 和 `/reset` 命令从管理员级别的 `sessions.reset` RPC 中拆分出来。**写权限范围**（Write-scoped）的 Gateway 客户端现在也可以重置对话，无需完整的管理员权限。这改善了 ACP 客户端和 IDE 插件等场景的用户体验。

本节小结

OpenClaw 的会话生命周期管理有以下核心特点：

1. **懒创建**：会话在第一条消息到达时自动创建，无需预先初始化
2. **双重过期机制**：每日重置（按固定时间点）和空闲重置（按不活跃时长）可以同时生效，取 OR 关系
3. **层次化覆盖**：`resetByChannel` > `resetByType` > `reset`，允许针对不同通道和会话类型设置不同策略
4. **手动触发器**：`/new` 和 `/reset` 命令提供即时重置能力，支持大小写不敏感匹配和附加消息
5. **运行时修补**：`sessions.patch` 允许动态修改会话属性，而 `sessions.reset` 在重置时保留用户偏好

下一节分析 OpenClaw 如何在 LLM 调用前裁剪过长的上下文，在有限的上下文窗口内保持对话质量。